

INDUSTRIAL SURFACE DEFECT INSPECTION SYSTEM

Phase 2 — Detailed Design, User Interface and Acceptance Test Plan

Team Name: VISION 404
Team Leader: Kanishk Rungta
Team Member: Nethi Kushala Kumar

1. Introduction

Phase 1 set out the problem, the requirements and the high-level architecture. Since then the inference pipeline has been built and measured. This report covers the application we build around that pipeline: how the software is put together, the database, the interface, the libraries we use, and how we plan to test it. It uses the requirement numbers from Phase 1 (FR-01, NFR-03 and so on). Section 2 lists everything from Phase 1 that has changed or been withdrawn.

2. Changes Since Phase 1

Building the pipeline and measuring it told us that several things in Phase 1 were either out of scope or not supported by evidence. Rather than leave those claims in the Phase 2 design, we have removed them.

What changed	Why
The derived fracture state is withdrawn.	It only existed to feed the verdict. The model outputs three classes — background, crack and scratch — and that is what we report.
The phone and ARM latency target (NFR-02) is withdrawn.	We never measured on ARM, so carrying the number would be an untested claim. The measured figure is 26 ms on one desktop CPU thread.
int8 quantisation is not delivered (NFR-01).	The exported model is 5.8 MB float32 with 1.43 M parameters, which is within the original 6 MB limit. The 2 MB int8 target is not met because that export does not exist yet.
SQLite is added as the queryable store.	Phase 1 planned flat files only. FR-18 needs filtering on six fields and FR-19 needs totals that reconcile, and doing that over flat files means writing our own query engine. SQLite is a single file with no server to install.

3. Detailed Design

3.1 System architecture

Four layers, each talking only to the one below. The inference core already exists as `app/inference.py` and has a fixed contract; the application layer and interface are what Phase 2 adds around it.

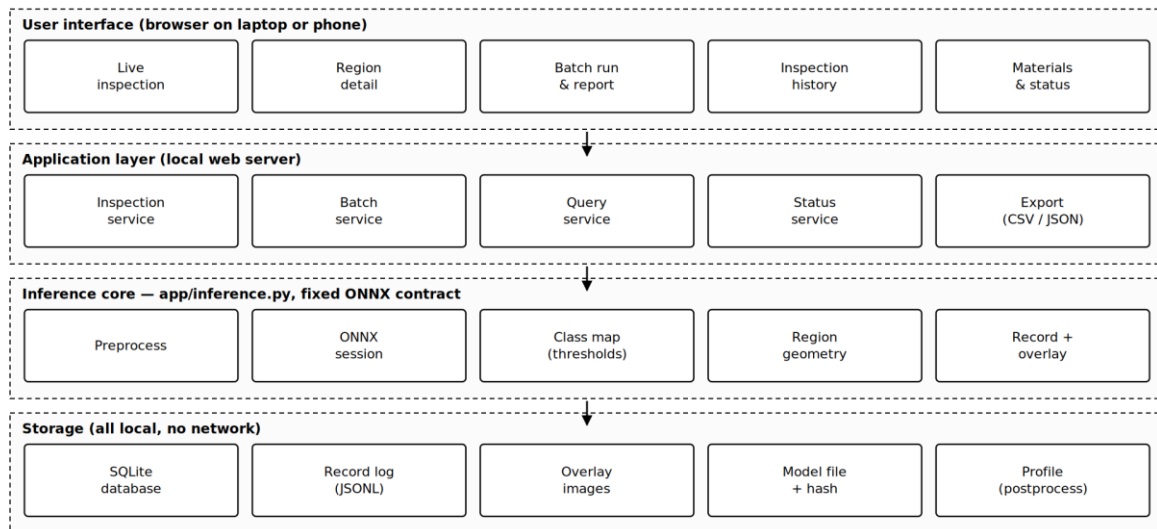


Figure 1. System architecture.

Everything goes through the `Inspector` class in `app/inference.py` rather than calling `onnxruntime` directly. The accuracy figures this project reports are the numbers that exact path produces, and a second implementation that resizes differently or uses `argmax` instead of the class thresholds would be a different system with different accuracy that no benchmark would catch.

3.2 Modules

Module	What it does	Input	Output
capture	Reads an image from the camera or from disk. Adds station ID, product ID and timestamp.	Camera or file	Image bytes + BGR array
inference.Inspector	The whole existing pipeline: preprocess, ONNX session, class map, geometry, record.	BGR image + bytes	Record, overlay, class map
preprocess	BGR to RGB, plain resize to 256×256, /255, ImageNet mean and std, CHW.	BGR array	float32 (1,3,256,256)
postprocess	Softmax, per-class thresholds, connected components, region geometry. Holds the Profile thresholds.	Logits	Class map + region list
overlay	Draws the masks and class labels on the source image at source resolution.	Image + class map	BGR overlay
writer	Appends the record to the log and inserts the database rows.	Record	Stored inspection
services	The local web API behind the five screens.	HTTP request	HTTP response

Module	What it does	Input	Output
batch	Runs the pipeline over a folder and produces a CSV or JSON report.	Folder path	Batch report
status	Checks camera, model file hash, database and disk space.	Config	Status report

3.3 Design flow

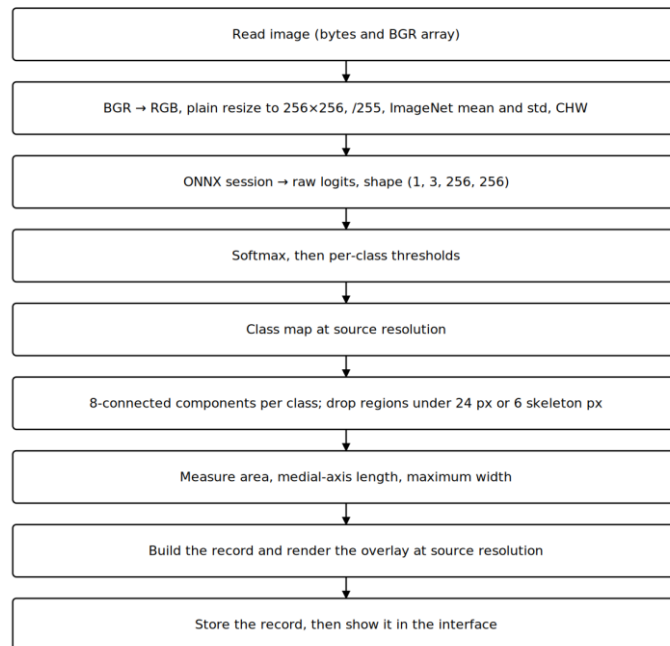


Figure 2. Flow of a single inspection.

Batch mode uses the same path; only the first step changes.

The class decision is not argmax. Each foreground class is compared against its own floor: a pixel below both floors is background, and if both pass, the higher score wins. Plain argmax is measurably worse — on the frozen factory split it gives crack recall 0.460 against 0.491 for the threshold rule at identical cIDice — and it has no background floor at all, so a pixel scoring 0.34 crack, 0.33 scratch and 0.33 background comes out as a confident crack.

The graph emits raw logits, not probabilities. Softmax is applied outside the graph. Feeding raw logits into a threshold meant for probabilities would appear to work while being badly miscalibrated, so the interface and any future consumer must go through the same path.

3.4 Model contract

The application is written against this contract rather than against a particular set of weights.

Input name and shape	input, (1, 3, 256, 256) float32, batch axis dynamic
Channel order	RGB, not BGR

Preprocessing	cv2.resize to (256, 256) with INTER_LINEAR, then /255, then ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225], then CHW
Output name and shape	logits, (1, 3, 256, 256) float32
Channel meaning	0 = red, 1 = green, 2 = blue
Activation	None in the graph. Softmax is applied by the caller.
Resize	Plain resize, not letterbox. Matching the training loader matters more than preserving aspect ratio, because preprocessing that differs between training and deployment is a silent accuracy loss.
Model file	1.43 M parameters, 5.8 MB float32, SHA-256 to be recorded and verified

The threshold values live in app/postprocess.py in the Profile class. The interface reads them from the module at start-up rather than holding its own copy, so the two cannot drift apart the next time the thresholds are re-tuned.

3.5 Region geometry

Measurement	How it is computed	Note
area_px	Pixel count of the connected component	8-connected components computed per class, so a crack touching a scratch stays two regions
length_px	Arc length along the medial axis, with diagonal steps counted as $\sqrt{2}$	Simply counting skeleton pixels would understate a diagonal defect by about 41%
max_width_px	Twice the maximum distance-transform value along the skeleton	This is the widest inscribable point, not the widest visual extent. A local bulge in a thin crack does not register, because the inscribed circle there is still limited by the crack's narrowness. The interface says so on the region detail screen.

Regions below 24 pixels of area or 6 skeleton pixels are dropped. Those floors come from the clean-surface false-positive budget, not from tuning against test images.

3.6 Material coverage

This is the honest state of the model, measured on the frozen split. It is shown in the interface as well as recorded here, because a type label on a material with no training data is transfer, not evidence.

Material	Status	Training masks	Class typing	Notes
Steel	Supported	Majority of the corpus	0.702	Best class typing, weakest geometry (cIDice 0.529) — steel cracks are the thinnest in the corpus at about 4 px
Plastic	One product only	936	—	All the real masks are PVC pipe. A moulded casing is not covered.
Ceramic	Thin coverage	120	0.315	Detection is usable, typing is weak

Material	Status	Training masks	Class typing	Notes
Epoxy	Typing unsupported	0	0.279	Detection works; the type label is transfer from other materials
Glass	Not supported	12	—	Not enough evidence to report anything
Non-steel metal	Not supported	0	—	No masks in the corpus

Scratch evidence is 624 of 724 masks on steel, so a scratch label on ceramic, plastic or glass is transfer as well.

Crack and scratch typing is the open defect in the project. The model detects about 96% of defects but assigns the correct type to only around 49% of crack pixels, biased towards scratch. The interface therefore treats the mask as the primary output and labels the class as provisional.

3.7 Database design

Eight tables. The point of the schema is that a stored inspection can always be explained later: which model file, which thresholds, which station and which image.

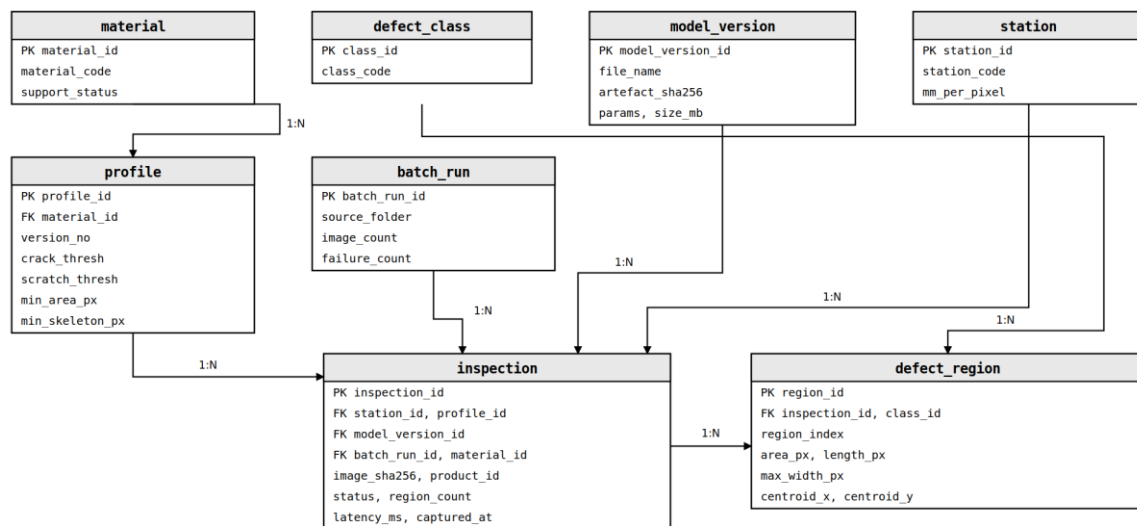


Figure 3. Database schema.

Table	Purpose	Key points
material	The materials in scope and their support status	PK material_id, unique material_code
defect_class	crack and scratch	PK class_id
model_version	An exported ONNX file	PK model_version_id, unique artefact_sha256, records parameter count and size

Table	Purpose	Key points
profile	The threshold set, versioned	PK profile_id, FK material_id, unique (material_id, version_no); mirrors Profile in app/postprocess.py
station	Camera station and its calibration	PK station_id, mm_per_pixel is null until the station is calibrated
batch_run	One batch job	PK batch_run_id, source folder, image and failure counts
inspection	One inspected image	PK inspection_id, FKs to station, model, profile, material and (optionally) batch run; stores image SHA-256, product ID, status and latency
defect_region	One detected region	PK region_id, FK inspection_id and class_id, unique (inspection_id, region_index)

The status column on inspection is the important one. It takes three values — regions_found, clean and acquisition_failure — because a defect-free product and an unreadable image are completely different things. Collapsing them into one “no defects” state would hide the difference between a good part and a broken camera.

Nothing is updated or deleted. Re-running an image produces a new inspection row rather than changing the old one, and threshold changes create a new profile version, so an old inspection still resolves to the thresholds that produced it.

3.8 Normalisation

Form	How the schema meets it
1NF	Every column holds a single value. The mistake to avoid is storing several regions inside the inspection row as a list; regions are a separate table keyed by (inspection_id, region_index).
2NF	Every table has a single-column primary key, so a partial dependency on part of a key cannot arise. defect_region uses a surrogate key with a unique constraint on (inspection_id, region_index).
3NF	No non-key column depends on another non-key column. The material name is not stored on inspection; the model file name and hash live only in model_version; threshold values live only in profile.

One deliberate exception: inspection.region_count is stored even though it could be counted from defect_region. It is there because the history and batch screens list thousands of rows and counting on every query is wasteful, and because the value can never change once the inspection is written.

3.9 Modularity

Change	What has to change
New weights after retraining	Swap the .onnx file and record the new hash. No code change unless input/output shape changes.
Swap ONNX Runtime for another CPU runtime	The session wrapper only. The model is a portable ONNX graph, not a runtime-specific format.

Change	What has to change
Re-tune thresholds	Can be reprocessed as per product.
Add a third defect class	Retrained model with a wider output head, one row in defect_class. The postprocessing loop is already per-class.
Add a verdict later, if tolerances become available	One new module between geometry and the record, plus a column on inspection. Nothing else is affected, because nothing downstream assumes a verdict exists.
Export results to a customer's system	One new endpoint in the services module.

3.10 Performance and scalability

The measured figure is 26 ms per 256×256 image on a single desktop CPU thread, with a 1.43 M parameter, 5.8 MB float32 model. Single-threaded on purpose: the deployment target is a CPU-only industrial PC that is also running line software, and a benchmark taken with every core free is not a number anyone can plan against.

Stage	Time
Read and decode	estimate, to be measured
Preprocess	estimate, to be measured
Model inference	26 ms (measured, 1 CPU thread, 256×256)
Postprocess and geometry	estimate, to be measured
Record and overlay	estimate, to be measured
End to end	T-19 measures this. Only the inference number exists today, and we are not going to quote a total we have not measured.

Aspect	Position
Faster lines	At 26 ms of inference there is headroom for roughly a part per second per station once the rest of the pipeline is measured. Beyond that, more worker processes on the same machine.
More stations	One unit per station. Units share nothing at runtime, so stations do not contend.
Reporting across stations	A separate machine can copy each unit's database file and record log on a schedule. The line never waits for it.
Database size	Roughly 1 KB per inspection including regions, so about 30 MB a day at one part per second. Older months are detached into archive files.
Image storage	Overlays and source images are kept for 48 hours when the result is clean and 90 days when regions were found. That is around 60 GB in steady state, which fits on a 1 TB SSD.
No ARM measurement	There is no ARM or phone figure, and we are not carrying one as a claim.

3.11 Security and data handling

There is no login and no user accounts, so this section is deliberately short. What the software can protect, it protects; what it cannot, we say so.

Concern	What we do
Factory images leaving the site	Nothing in the inference path opens a socket — onnxruntime on CPU, a local weights file, and no client library that could phone home. We test this by running a full cycle with the network disabled (T-20).
The model file being swapped	The SHA-256 of the .onnx file is recorded in model_version and verified when it is loaded. A mismatch stops the run rather than producing results from an unknown model.
Records being changed after the fact	Nothing in the schema is updated or deleted. Re-processing produces a new row.
Bad files in a batch folder	Extension and magic-byte check, a decode size limit, and paths resolved so they cannot escape the configured folder.
Disk filling up	Free space is a status check. Below the threshold the run stops rather than dropping records silently.
Who did what	The system cannot tell you. There are no accounts, so an action cannot be attributed to a person. That is a consequence of dropping the verdict and the override workflow, and it is a real limitation — if a customer later needs attribution, accounts would have to come back.

4. User Interface

4.1 What the interface shows, and what it does not

Most of the interface design follows from three rules about what must not appear on screen.

Rule	Reason
Do not show a confidence percentage.	The scores are uncalibrated. Rendering 0.87 as “87% confident” states a probability the model has never been validated to produce.
Do distinguish clean from failed to process.	A defect-free product and an unreadable image are different outcomes. Showing both as “no defects” hides the difference between a good part and a broken camera.
Do show the overlay, the region list and the geometry.	That is the evidence. It is what the operator actually needs to make a judgement.
Do label the class as provisional.	Typing is only about 49% correct on crack pixels (Section 3.6), so the mask is the primary output and the class label is secondary.

Beyond those, the interface is read-only, station state is visible on every screen, the result has to be readable in about two seconds, and the same layout works on a laptop and a phone.

4.2 Screens

Screen	Purpose	Main user
Live inspection	The result for the item currently at the station	Operator

Screen	Purpose	Main user
Region detail	Look at each region closely with its measurements	Operator, quality engineer
Batch run and report	Run the pipeline over a folder and export the results	Quality engineer
Inspection history	Search past inspections and open the original evidence	Quality engineer
Materials and thresholds	Measured coverage per material, active thresholds, model in use	Quality engineer
Status	Camera, model hash, database and disk checks	Anyone

4.3 Live inspection

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Live Inspection — line-1-cam-A

Camera OK | Model v8a 5.8 MB hash 9f2c... | Profile: steel | Disk 71% | 26 ms

2

overlay at source resolution

Product batch-77 / item-12 11:42:07.318 steel

3 REGIONS FOUND

2 crack, 1 scratch

Regions

#	Class	Length	Width	Area
1	crack	212 px	4 px	684
2	crack	409 px	9 px	2,118
3	scratch	156 px	11 px	1,402

Class label is provisional — see Materials for measured coverage by material.

Open region detail

View history

Figure 4. Live inspection screen.

1. Status strip: camera, model file and hash, active profile, disk and last latency. Visible on every screen.
2. The overlay is drawn at source resolution, not at 256×256, so it lines up with the actual part.
3. The summary states what was found — a count and a class breakdown. It is not a verdict. The other two states are ‘NO DEFECTS FOUND’ and ‘COULD NOT PROCESS’.
4. The region table is the evidence: class, length, maximum width and area per region. No score is shown.
5. The class label is marked provisional and links to the coverage table on the Materials screen.

4.4 Region detail

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Region Detail — inspection 00184

Regions (3)

1 crack 212 px

2 crack 409 px

3 scratch 156 px

< Prev

Next >

cropped region, zoomable

Crop at 3x zoom | overlay on / off / side-by-side

Measurements

Class crack (provisional)

Area 2,118 px

Length 409 px / 8.18 mm

Max width 9 px / 0.18 mm

Length is medial-axis arc length. Max width is the widest inscribable point, not the widest visual extent.

Source

Image sha256 9f2c...a41

Model v8a, float32

Profile steel v2 — crack ≥ 0.40 , scratch ≥ 0.20 , min area 24, skeleton 6

Figure 5. Region detail screen.

1. The region list makes the number of things to look at explicit. Previous and next move through them.
2. Only the cropped region is enlarged, which is a much lighter task than scanning a whole surface.
3. The measurements panel states how length and maximum width are defined, including the limitation that maximum width is the widest inscribable point rather than the widest visual extent.
4. The source panel records the image hash, the model file, the profile version and the exact thresholds used, so the result can be reproduced.

4.5 Batch run and report

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Batch Run and Report

Run configuration

Source folder

/mnt/captures/2026-08-12/steel-lineB

Material

▼ steel

Model file

▼ v8a (float32)

Start run

Dry run

312 / 500 images | est. 00:23 remaining

Processed

312

of 500

With regions

44

14.1%

Clean

264

85.9%

Failed to read

4

excluded

Regions found

97

68 crack / 29 scratch

Results

Image	Product ID	Status	Regions	Classes	Max length
IMG_0311.png	batch-77/item-11	clean	0	—	—
IMG_0312.png	batch-77/item-12	regions found	3	2 crack, 1 scratch	409 px
IMG_0313.png	batch-77/item-13	regions found	1	1 scratch	156 px
IMG_0314.png	—	failed to read	—	—	—

Export CSV

Export JSON

Only images with regions

Figure 6. Batch run and report screen.

1. Summary cards are calculated from the same rows the export produces, so the totals always reconcile.
2. Clean, regions found and failed to read are three separate statuses. A file that could not be read is never counted as clean.
3. The filter gives the quality engineer just the images that produced regions.

4.6 Inspection history

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Inspection History

Filters

Date from / to

Material

Status

Class

Station

Product ID

Apply

Clear

01 Aug - 12 Aug

▼ steel

▼ regions found

▼ crack

▼ cam-A

batch-77/*

1,284 matching inspections

Timestamp	Product ID	Material	Status	Regions	Max length	Model
12 Aug 11:42:07	batch-77/item-12	steel	regions found	3	409 px	v8a
12 Aug 11:41:58	batch-77/item-11	steel	clean	0	—	v8a
12 Aug 11:41:49	batch-77/item-10	steel	regions found	1	156 px	v8a
12 Aug 11:41:40	batch-77/item-09	steel	failed to read	—	—	v8a
12 Aug 11:41:31	batch-77/item-08	steel	regions found	2	301 px	v7

< Previous 1 2 3 ... 26 Next >

Export CSV

Open selected

Figure 7. Inspection history screen.

1. All the filters from FR-18 are on one row. There is no hidden advanced search.
2. Opening a row shows the original overlay, the measurements, the model version and the thresholds used for that inspection.

4.7 Materials and thresholds

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Materials and Thresholds

Measured coverage — read-only

Material	Status	Training masks	Class typing	Notes
Steel	supported	majority of corpus	0.702	thinnest cracks (~4 px); cIDice 0.529
Plastic	one product only	936	—	all masks are PVC pipe
Ceramic	thin coverage	120	0.315	detection usable, typing weak
Epoxy	typing unsupported	0	0.279	detection works, type label is transfer
Glass	not supported	12	—	insufficient evidence
Non-steel metal	not supported	0	—	no masks in corpus

Active thresholds (from app/postprocess.py)

crack_thresh 0.40

scratch_thresh 0.20

min_area_px 24

min_skeleton_px 6

Read from the module at start-up, not copied into the UI.

Model in use

smpslim_mobilenetv3_small_v8a.onnx

sha256 9f2c...a41 (verified at load)

1.43 M parameters, 5.8 MB float32

26 ms per image, 1 CPU thread

No ARM or phone figure exists.

Figure 8. Materials and thresholds screen.

1. The coverage table is read from the metrics file produced by evaluation, not typed into the interface, so it cannot drift from the measured numbers.
2. Thresholds are read from app/postprocess.py at start-up and shown read-only. They are not copied into the interface.
3. The model panel shows the file, its hash, its size and the measured latency, including the note that no ARM figure exists.

4.8 Status screen

The status screen shows four checks — camera, model file hash, database and disk space — with the last check time and a button to run them again. Any check in a non-OK state is reflected in the status strip on every other screen, and a failed model hash check stops inspection rather than letting the system produce results from a file it cannot identify.

4.9 Result states

State	Shown as	What it means
Regions found	‘n REGIONS FOUND’ with a class breakdown, grey banner	The model found regions. The operator decides what to do with them.
Clean	‘NO DEFECTS FOUND’, white banner	Recorded with an empty region list and status clean
Could not process	‘COULD NOT PROCESS’, hatched banner, names the reason	An acquisition failure. Never shown as clean.
Station problem	‘CHECK STATION’, dark banner, names the failing check	Camera, model hash, database or disk. Inspection is stopped.

Each state is distinguished by the wording, the fill and the border together, so the screen still reads under glare and for a colour-blind user.

5. Third-Party Libraries and Tools

Everything we use is free and open source. There is no paid library, licence key or cloud service anywhere in the system. Phase 1 found that high initial investment is one of the main reported barriers to adopting this kind of system in Indian manufacturing [R3, R4], so the software licence cost of the deployed system is zero and the only cost is the industrial PC and the camera.

5.1 Libraries that run on the inspection PC

Library	Version	Used for	Licence	Cost
Python	3.11	Runtime	PSF	Free
ONNX Runtime	1.18	Running the model on CPU	MIT	Free
NumPy	1.26	Arrays for tensors, class maps and geometry	BSD-3	Free
OpenCV (headless)	4.10	Decode, resize, connected components, drawing the overlay	Apache-2.0	Free
scikit-image	0.24	Medial axis and distance transform for length and width	BSD-3	Free
SQLite (Python stdlib)	3.45	Database	Public domain	Free
FastAPI + Uvicorn	0.111 / 0.30	Local web API and server	MIT / BSD-3	Free
Jinja2	3.1	HTML templates	BSD-3	Free

There is no JavaScript framework and no build step. The interface is server-rendered HTML with a few hundred lines of plain JavaScript for image zoom and refreshing the live result. Because there is no login, there is also no authentication or password-hashing library.

5.2 Libraries used only for training and development

Library	Used for	Licence
PyTorch, timm	Training and the pretrained MobileNetV3-Small backbone	BSD-3 / Apache-2.0
albumentations	Data augmentation	MIT
onnx, onnxruntime-tools	Exporting the graph and checking it	Apache-2.0 / MIT
ImageHash	Perceptual-hash leakage check across dataset splits	BSD-2
pandas, matplotlib	Dataset manifests, metric tables and plots	BSD-3 / Matplotlib
pytest, pytest-cov	Test runner and coverage	MIT
Playwright	Browser automation for interface tests	Apache-2.0
ruff, mypy	Linting and type checking	MIT
Git, GitHub Actions	Version control and continuous integration	Free tier

5.3 How much of the system is third-party

By line count almost all of it, which is true of any Python project and is not a useful number on its own. By capability it splits like this.

Capability	Source
Image decode, resize, connected components	Third-party (OpenCV)
Running the neural network	Third-party (ONNX Runtime)
Medial axis and distance transform	Third-party (scikit-image)
HTTP transport and templating	Third-party (FastAPI, Jinja2)
Model architecture, training and export	Ours, on top of PyTorch and timm
Preprocessing that matches the training loader exactly	Ours
The per-class threshold rule instead of argmax	Ours
Region floors and the geometry definitions	Ours
Record format, database schema and retention	Ours
The entire interface	Ours
Batch runner and reporting	Ours
Dataset governance and leakage checks	Ours

So roughly a third of the capabilities are library work, and they are the general-purpose parts — arithmetic, image handling and transport. The parts that decide whether the output is

trustworthy are ours: what counts as a region, how a class is assigned, what gets measured and how, and what the interface refuses to claim.

5.4 Dependency risk

Dependency	If it went away
ONNX Runtime	Our biggest dependency. The model is a portable ONNX graph, so OpenVINO or TFLite could run it; only the session wrapper changes.
OpenCV	Pillow and scikit-image cover decode, resize and connected components, but slower, so latency would need rechecking. The resize would also have to reproduce INTER_LINEAR exactly or accuracy would shift.
scikit-image	Medial axis and distance transform are standard algorithms we could implement ourselves.
FastAPI, Uvicorn, Jinja2	Flask, Waitress and plain templates. The service layer is thin.
SQLite	Not a realistic risk — public domain and shipped inside Python.

5.5 Licences

All the runtime licences are permissive — MIT, BSD, Apache-2.0, PSF or public domain. We will ship a notices file with the copyright and licence text and did not modify any of the libraries, so there is no obligation to publish our own source.

The real constraint is on data, not code. KolektorSDD2 and MVTec AD are CC BY-NC-SA 4.0, which is non-commercial, and the Severstal data comes under Kaggle competition terms. That is fine for a prototype and for this hackathon, but a commercial version would have to retrain on licensed or self-collected images.

6. Acceptance Test Plan

6.1 Approach

Phase 1 listed twenty-two acceptance tests as intentions. A number of new cases exist because of things we learned building the pipeline: that preprocessing must match training exactly, that the class rule is not argmax, and that clean and failed-to-read must not be confused.

Unit and integration tests run on every commit. The twenty-five acceptance cases below run at each project gate and in full before submission.

6.2 Test data

Set	What it is
Frozen factory split	The real-image test split, sealed and opened only for the final measurement. Its manifest hash is recorded.
Clean-surface set	Real defect-free images, never used in training, for measuring false-positive area.
Synthetic shapes	Lines and arcs with known area, length and width, including diagonals, for testing measurement rather than detection.

Set	What it is
Score-map fixtures	Hand-built logit arrays where argmax and the threshold rule disagree, and where all three classes score near 0.33.
Unreadable files	Truncated PNG, zero-byte file, oversized image, and a text file with a .png extension.
Reference targets	Printed lines of known length and width, used on the physical rig for the millimetre conversion.

6.3 Test cases

ID	What we do	Pass criterion
T-01	Run 200 valid images through Inspector.	All 200 produce a record; no unhandled exception; each record carries station ID, product ID and timestamp.
T-02	Compare the deployed preprocessing against the training loader on 50 images, element by element.	The tensors match within float tolerance. Any difference in resize, scaling or normalisation fails the test.
T-03	Inspect the ONNX graph: input and output names, shapes, dtype, and whether the output is bounded.	Input is 'input' (1,3,256,256) float32; output is 'logits' with the same shape; values outside [0,1] are present, confirming raw logits.
T-04	Run the frozen split and collect every value in the class map.	Only 0, 1 and 2 appear.
T-05	Feed the score-map fixtures where argmax and the threshold rule disagree.	The rule follows the per-class floors: below both floors is background, and where both pass, the higher score wins. A 0.34 / 0.33 / 0.33 pixel comes out as background, not a crack.
T-06	Change Profile.crack_thresh in app/postprocess.py and re-run the pipeline and the interface.	Both reflect the new value with no other edit. No threshold value is hard-coded anywhere in the interface.
T-07	Run the clean-surface set.	At least 98% return an empty region list with status clean; false-positive area at or below 0.5%.
T-08	Submit a clean image and each of the unreadable files.	The clean image is status clean with an empty region list; each bad file is an acquisition failure. Neither is ever reported as the other.
T-09	Measure the synthetic shapes, including a 45° line.	Length within $\pm 3\%$ and maximum width within ± 1 px of the true value. The diagonal is not understated, confirming the $\sqrt{2}$ step.
T-10	Measure a thin crack with a local bulge.	The reported maximum width is the inscribable width. The test records the behaviour and the interface note is present; we do not assert the visual width.

ID	What we do	Pass criterion
T-11	Build regions of 20 px and 30 px area, and of 4 and 8 skeleton pixels.	Regions below either floor are dropped and those above are kept.
T-12	Measure a fixture where a crack and a scratch touch.	Two separate regions are returned, one per class, each with its own geometry.
T-13	Compare the overlay and class map dimensions with the source image.	Both are at source resolution, not 256×256, and the overlay registers to the source.
T-14	Read every field of 100 stored records.	Image SHA-256, model version and artefact hash, profile version, station and product ID are present on all of them.
T-15	Run the same 100 images twice with the model and profile fixed.	Identical class maps, identical geometry and identical record fields apart from the timestamp. Golden fixtures match.
T-16	Run a batch of 500 images, export CSV and JSON, and recalculate the totals independently.	One row per region plus one per clean image; failures carry a status; exported totals match the recalculated ones.
T-17	Query the history on each filter separately and then in combination.	Each result set matches an independently computed set, including when the answer is empty.
T-18	Evaluate the frozen split per class and per material.	clDice, tolerant F1 and IoU reported per material; detection and class typing reported separately, never merged into one number.
T-19	Measure model size and per-image latency over 500 runs on one CPU thread, then measure the full end-to-end path.	5.8 MB float32; inference median at or below 30 ms; the end-to-end figure is recorded with the thread count stated. No ARM figure is quoted.
T-20	Disable the network and run a live inspection and a batch run, tracing system calls.	Both finish and no socket is opened at any point.
T-21	Search every rendered page for a percentage, a score, or accept/reject wording.	No probability or confidence value appears anywhere, and no verdict wording appears anywhere.
T-22	Compare the coverage table on the Materials screen against the metrics file from evaluation.	Every number on screen comes from the metrics file. No coverage figure is hard-coded.
T-23	Replace the .onnx file with a different one of the same name and start the application.	The hash mismatch is detected at load, inspection does not start, and the status screen names the problem.
T-24	Put synthetic and unreviewed samples into the validation and test splits, then run the dataset QA gate and the leakage check.	The gate fails and names the offending files; no near-duplicates cross a split boundary.
T-25	On the rig: photograph the printed reference targets at the calibrated distance, then run a timed walkthrough with five users.	Length within ±5% and width within ±1 px; the millimetre conversion matches the station calibration; all fields visible without opening logs and a region reviewed in no more than three clicks.

6.4 Automation

Twenty-four of the twenty-five cases run automatically. Only T-25 is manual, because it needs a physical target in front of a real camera and actual people doing a timed task.

The automated cases run in GitHub Actions on every merge: linting and type checks, then unit tests, then integration tests on the pipeline, then the acceptance cases against the frozen fixtures. A failure at any stage blocks the merge. Most of this is automatable because the pipeline takes an image array in and returns a record out — no camera, no database and no GPU are needed to run it.

When a bug is found we add a test that fails before the fix and passes after it, and it stays in the suite. Golden fixtures are only regenerated by a deliberate committed change; a fixture that regenerates itself makes the repeatability test meaningless.

6.5 Traceability

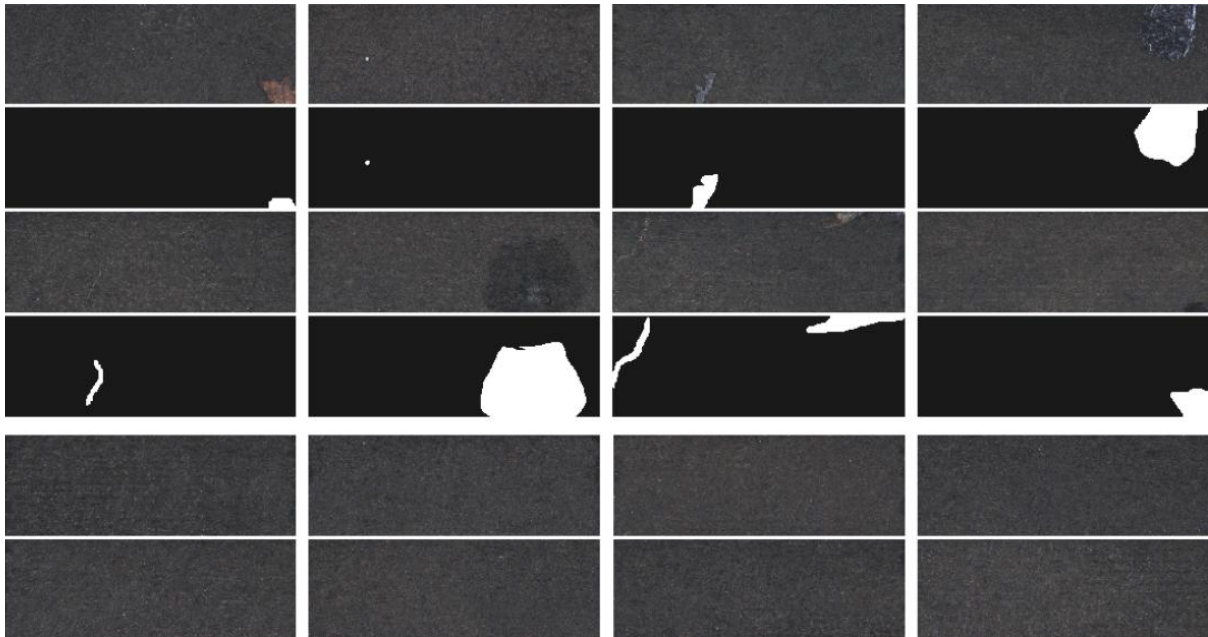
Requirement	Covered by	Requirement	Covered by
FR-01	T-01, T-08	NFR-01	T-19
FR-02	T-08	NFR-02	T-19
FR-03	T-02	NFR-03	T-07
FR-04	T-13	NFR-05	T-20
FR-05	T-04, T-05, T-12	NFR-06	T-15
FR-06	T-07	NFR-07	T-24
FR-07	T-09, T-10, T-11, T-25	NFR-08	T-18
FR-08	T-13	NFR-09	T-21
FR-09	T-16	NFR-10	T-08
FR-10	T-14	NFR-12	T-18
FR-11, FR-12	T-24	NFR-13	T-23
FR-13	T-22	NFR-14	T-25
FR-17	T-25	Model contract Section 3.4	T-02, T-03, T-06
FR-18	T-17		
FR-19	T-16		

6.6 Defect handling

Severity	Meaning	What we do
S1	A test fails in a way that means the output could be wrong without anyone noticing — preprocessing drift,	Stop other work, fix it, and re-run the whole group.

Severity	Meaning	What we do
	the class rule not being applied, or clean and failed-to-read being confused.	
S2	A test fails, or a requirement only works under restricted conditions.	Fix within the current work package, or write the restriction into the limitations list.
S3	Cosmetic or usability issue with no effect on the output.	Backlog, fix if there is time.

7. Acceptance Test Cases

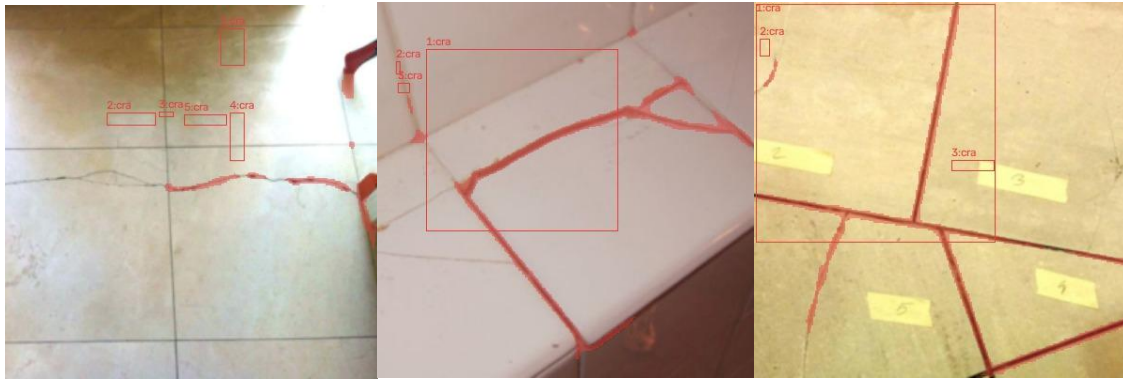


The first 2 rows show the materials with cracks/scratches of which the model segments that particular area as output. The final row is of accepted materials with no defects.

Input:



Output:



8. Other Observations

8.1 What we learned

The most useful thing that came out of building the pipeline was finding out which Phase 1 claims we could not support. The fracture state, the override workflow and the whole login and roles design went with it. The system got smaller and more honest at the same time.

The second thing is that the details that decide whether the output is correct are unglamorous ones. Preprocessing that differs between training and deployment loses accuracy silently. Using argmax instead of the per-class floors loses three points of crack recall and turns a 0.34 / 0.33 pixel into a confident detection. Neither of those would show up in any metric we report, which is why T-02, T-03 and T-05 exist.

8.2 Known limitations

Limitation	Our position
Class typing is only about 49% correct on crack pixels.	This is the main open defect. Detection is around 96%, so the mask is the useful output. The interface marks the class as provisional and we say so here rather than in a footnote.
Maximum width is the widest inscribable point.	A local bulge in a thin crack does not register. The interface states this on the region detail screen rather than letting an operator assume otherwise.
Only the inference stage of the latency is measured.	26 ms on one CPU thread. The end-to-end figure comes from T-19; we are not quoting a total we have not measured.
No ARM or phone measurement.	The requirement was withdrawn rather than carried as an untested claim.
No int8 model.	The 5.8 MB float32 export is within the original 6 MB limit; the 2 MB int8 target is not met.
No user accounts, so no attribution.	A deliberate consequence of dropping the verdict. If a customer needs to know who did what, accounts would have to come back.
Two datasets are non-commercial.	Fine for the prototype, blocking for a product.

Limitation	Our position
Subsurface flaws are not detected.	The system is optical and does not replace ultrasonic, radiographic or penetrant testing.

8.3 Risks

Risk	Mitigation
Class typing does not improve enough to be useful	The mask is already the primary output, so the product still works with a provisional class label. More masks on non-steel materials is the only real fix, and it is the top data priority.
Someone reimplements the preprocessing in the UI layer and it drifts from training	Everything goes through Inspector, and T-02 compares the deployed preprocessing against the training loader element by element.
Thresholds get copied into the interface and then re-tuned in the module	The interface reads them from app/postprocess.py at start-up; T-06 checks this by changing the module value.
Not enough real imagery for the non-steel materials	Coverage is reported per material and shown in the interface. We would rather say 'not supported' than report a number we cannot back.
Two-person team capacity	The database and interface are the only new code in Phase 2; the pipeline already exists. Plain SQL rather than an ORM so there is less to maintain.